

**Федеральное государственное автономное образовательное учреждение высшего
образования «Национальный исследовательский университет ИТМО»**

Факультет программной инженерии и компьютерной техники

ОТЧЁТ
по задачам: Е-Н
по дисциплине «Алгоритмы и структуры данных»

Выполнил:
Ануфриев Андрей Сергеевич
Р3219

Содержание

1	Задание Е. Коровы в стойла	2
2	Задание F. Число	4
3	Задание G. Кошмар в замке	5
4	Задание H. Магазин	7

1 Задание Е. Коровы в стойла

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> stalls(n);
    for (int i = 0; i < n; i++) {
        cin >> stalls[i];
    }
    int left = 0;
    int right = stalls[n - 1] - stalls[0];
    int answer = 0;
    while (left <= right) {
        int mid = (left + right) / 2;
        int cows = 1;
        int last_pos = stalls[0];
        for (int i = 1; i < n; i++) {
            if (stalls[i] - last_pos >= mid) {
                cows++;
                last_pos = stalls[i];
            }
        }
        if (cows >= k) {
            answer = mid;
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    cout << answer << endl;
    return 0;
}
```

Суть решения

Ключевая идея: использование бинарного поиска по ответу. Ищем нужное расстояние между коровами, которое можно достичь при расстановке k коров в n стойл.

Основной алгоритм:

1. **Инициализация бин.поиска:** диапазон поиска от 0 до (последнее стойло - первое стойло).
2. **Бин.поиск:** для каждого проверяемого расстояния mid проверяем возможность расстановки.
3. **Жадно расставляем:**
 - первую корову на первое стойло.
 - Для следующего стойла: если расстояний $\geq mid$, ставим туда и обновляем $last_pos$.

- Если удалось расставить, увеличиваем `mid`.
- Если не удалось, уменьшаем.

Реализация простого бин.поиска

Время: $O(n \log(\max_dist))$

Память: $O(n)$

2 Задание F. Число

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main() {
    vector<string> input;
    string s;
    while (cin >> s) {
        input.push_back(s);
    }
    sort(input.begin(), input.end(), [](const string &a, const string &b) {
        return a + b > b + a; });
    string result;
    for (const string &piece: input) {
        result += piece;
    }
    cout << result << endl;
    return 0;
}
```

Суть решения

Ключевая идея: делаем красивую сортировку чисел как букв

Почему это работает: если $a+b > b+a$, то в любом числе, где a и b стоят рядом, a должна быть перед b для максимизации.

Время: $O(n \log(n L))$ - где n - количество фрагментов, L - средняя длина фрагмента.

Память: $O(S)$

3 Задание G. Кошмар в замке

```
#include <algorithm>
#include <iostream>
#include <map>
#include <string>
#include <vector>
using namespace std;

int main() {
    string s;
    cin >> s;
    vector<long long> weights(26);
    for (int i = 0; i < 26; i++) {
        cin >> weights[i];
    }
    int n = s.length();
    vector<int> freq(26, 0);
    for (char c: s) {
        freq[c - 'a']++;
    }
    vector<pair<long long, char> > letters;
    for (int i = 0; i < 26; i++) {
        if (freq[i] > 0) {
            letters.push_back({weights[i], char('a' + i)});
        }
    }
    sort(
        letters.begin(),
        letters.end(),
        [](const pair<long long, char> &a, const pair<long long, char> &b) {
            return a.first > b.first;
        }
    );
    vector<char> result(n);
    int left = 0, right = n - 1;
    vector<char> middle;
    for (auto &p: letters) {
        char c = p.second;
        int count = freq[c - 'a'];
        if (count == 1) {
            middle.push_back(c);
        } else if (count >= 2) {
            result[left++] = c;
            result[right--] = c;
            for (int i = 2; i < count; i++) {
                middle.push_back(c);
            }
        }
    }
    int mid_index = left;
    for (char c: middle) {
        result[mid_index++] = c;
    }
    for (char c: result) {
```

```
    cout << c;  
}  
cout << endl;  
return 0;  
}
```

Суть решения

Ключевая идея: для каждой буквы с частотой > 1 , максимальное расстояние между её вхождениями достигается, когда одно вхождение находится в начале строки, а другое - в конце. Буквы с большим весом должны быть размещены на максимальные расстояния.

Основной алгоритм:

1. Подсчет частот и чтение весов
2. Создание списка букв
3. Сортировка по убыванию веса
4. Используем два указателя: left и right
5. и расставляем буквы с частотой > 1 по краям, а остальные в середину.

Я с начала думал, что учитывается вес каждой буквы (каждого вхождения) и пытался как-то расставлять буквы, которые встречаются > 2 раз. Но потом потыкавшись в тесты, я понял, что надо внимательнее читать задание.

Время: $O(n + k \log k)$ - где n - длина строки, $k \leq 26$

Память: $O(n)$

4 Задание Н. Магазин

```
#include <algorithm>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;
    vector<int> nums(n);
    for (int i = 0; i < n; i++) {
        cin >> nums[i];
    }
    sort(nums.begin(), nums.end(), greater<int>());
    long result = 0;
    for (int i = 0; i < n; i++) {
        if ((i + 1) % k != 0) {
            result += nums[i];
        }
    }
    cout << result << endl;
    return 0;
}
```

Суть решения

Ключевая идея: жадный алгоритм с сортировкой. **Основной алгоритм:**

1. Сортировка в порядке убывания
2. Подсчет суммы с пропусками

Время: $O(n \log n)$

Память: $O(n)$